Embedded Software Project: STM32 Peripheral Drivers

This project focuses on developing and testing bare-metal peripheral drivers for the STM32F401RE microcontroller. The drivers provide hardware abstraction layers for GPIO, USART, I2C, and SPI peripherals, allowing application programs to interact with hardware without directly accessing registers. Each driver configures the required clock, registers, and communication parameters through dedicated APIs. The developed drivers were tested using practical examples to verify GPIO control, serial communication, and peripheral interfacing.

GPIO Driver Analysis

The GPIO driver is used to control the General Purpose Input Output (GPIO) pins of the STM32F401RE microcontroller. Instead of writing directly to the GPIO registers in every application, the driver provides simple API functions that make GPIO programming easier and more reusable.

The first function in the driver is GPIO_PeriClockControl(). This function enables or disables the clock for the required GPIO port. Before configuring any GPIO pin, the peripheral clock must be enabled through the RCC AHB1 peripheral clock register. The function checks which GPIO port (GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, or GPIOH) is selected and enables or disables its clock accordingly.

The GPIO_Init() function is responsible for configuring a GPIO pin. It allows the user to select the pin mode such as input, output, alternate function, analog mode, or interrupt mode. It also configures the output speed, output type (push-pull or open-drain), and pull-up/pull-down resistor settings. If the pin is used in alternate function mode, the required alternate function value is written into the AFR register. For interrupt mode, the function configures the EXTI and SYSCFG registers and enables the corresponding interrupt line.

The GPIO_DeInit() function resets the selected GPIO peripheral using the RCC reset register. This clears all previous configurations and restores the GPIO peripheral to its default state.

The driver also provides functions to read and write GPIO pins. GPIO_ReadFromInputPin() reads the logic level of a single input pin, while GPIO_ReadFromInputPort() reads the complete input port. Similarly, GPIO_WriteToOutputPin() writes HIGH or LOW to a selected output pin, and GPIO_WriteToOutputPort() writes data to the entire output port. The GPIO_ToggleOutputPin() function changes the current state of the selected output pin and is mainly used for applications like LED blinking.

For interrupt handling, the driver includes GPIO_IRQInterruptConfig(), GPIO_IRQPriorityConfig(), and GPIO_IRQHandling(). These functions enable or disable GPIO interrupts, configure interrupt priority in the NVIC, and clear the interrupt pending flag after the interrupt is serviced.

In conclusion, GPIO driver provides a simple interface for configuring and controlling GPIO pins without directly accessing hardware registers in the application code. It makes the code more modular, reusable, and easier to understand. The driver supports GPIO input, output, alternate function, analog mode, and external interrupts, making it suitable for most embedded

system applications. The GPIO APIs were successfully tested on the STM32F401RE Nucleo board by controlling the onboard LED and configuring GPIO pins as inputs and outputs.

SPI Driver Analysis

The SPI driver is developed to provide a simple interface for configuring and communicating with the SPI peripheral of the STM32F401RE microcontroller. Instead of directly accessing the SPI registers in the application, the driver provides API functions for initialization, data transmission, data reception, and peripheral control. This makes the application code more modular and easier to understand.

The SPI_PeriClockControl() function is used to enable or disable the clock for the SPI peripheral. It checks whether SPI1, SPI2, or SPI3 is selected and enables or disables the corresponding peripheral clock through the RCC registers. Since the STM32F401RE supports SPI1, SPI2, and SPI3, these peripherals were included in the driver. The clock must be enabled before configuring or using any SPI peripheral.

The SPI_Init() function is responsible for configuring the SPI peripheral. It first enables the SPI clock and then configures the required communication parameters. The function allows the user to select the device mode (Master or Slave), communication mode (Full Duplex, Half Duplex, or Simplex Receive), serial clock speed (baud rate), data frame format (8-bit or 16-bit), clock polarity (CPOL), clock phase (CPHA), and software slave management (SSM). All these settings are written to the SPI control register (CR1), allowing the SPI peripheral to operate according to the selected configuration.

The SPI_DeInit() function resets the selected SPI peripheral using the RCC reset register. This clears all previous configurations and restores the peripheral to its default reset state.

The SPI_GetFlagStatus() function checks the status of SPI flags by reading the status register (SR). It is mainly used to monitor the TXE (Transmit Buffer Empty) and RXNE (Receive Buffer Not Empty) flags before transmitting or receiving data.

The driver provides two blocking communication APIs: SPI_SendData() and SPI_ReceiveData(). The SPI_SendData() function waits until the TXE flag is set before writing data into the data register (DR). It supports both 8-bit and 16-bit data frame formats. Similarly, the SPI_ReceiveData() function waits until the RXNE flag is set before reading data from the data register. Since both functions continuously check the status flags before transferring data, they use a polling (blocking) method instead of interrupts.

While porting the driver to the STM32F401RE, the driver was updated to use the STM32F401RE device header file and board-specific definitions. Only the SPI peripherals available on the STM32F401RE (SPI1, SPI2, and SPI3) were configured. The driver was tested by initializing the SPI peripheral and performing transmit and receive operations using the provided APIs.

Overall, the SPI driver provides a simple and reusable interface for SPI communication without requiring the application to access hardware registers directly. It supports different SPI communication modes, configurable clock settings, 8-bit and 16-bit data transfers, and

blocking data transmission and reception, making it suitable for embedded communication with SPI-based devices.

I2C Driver Analysis

The I2C driver is developed to provide an easy interface for communication between the STM32F401RE microcontroller and I2C devices such as sensors, EEPROMs, and displays. Instead of directly accessing the I2C registers in the application program, the driver provides API functions for peripheral initialization, data transmission, and data reception. This improves code readability and makes the driver reusable for different I2C devices.

The I2C_PeriClockControl() function is used to enable or disable the clock for the I2C peripheral. It checks whether I2C1, I2C2, or I2C3 is selected and enables or disables the corresponding peripheral clock through the RCC registers. Since the STM32F401RE supports these three I2C peripherals, the driver was modified to configure only the peripherals available on this microcontroller.

The I2C_Init() function is responsible for configuring the I2C peripheral. It first enables the peripheral clock and then configures important communication parameters such as ACK control, peripheral clock frequency (CR2), device own address (OAR1), clock control register (CCR), and maximum rise time (TRISE). The CCR value is calculated based on the APB1 clock frequency and the selected I2C speed, allowing the peripheral to operate in either Standard Mode (100 kHz) or Fast Mode (400 kHz). The TRISE register is also configured according to the selected communication mode to ensure proper signal timing.

The driver also contains helper functions such as I2C_GenerateStartCondition(), I2C_GenerateStopCondition(), I2C_ExecuteAddressPhaseWrite(), I2C_ExecuteAddressPhaseRead(), and I2C_ClearADDRFlag(). These functions generate the START and STOP conditions, send the slave address with the appropriate read or write bit, and clear the ADDR flag after address matching. These helper functions simplify the implementation of the main communication APIs.

The I2C_GetFlagStatus() function checks the status of different I2C flags by reading the SR1 register. It is used to monitor important events such as START condition generation (SB), address matching (ADDR), transmit buffer empty (TXE), receive buffer not empty (RXNE), and byte transfer finished (BTF).

The I2C_MasterSendData() function implements blocking (polling-based) data transmission. It generates the START condition, sends the slave address in write mode, waits for the required status flags, transfers the data bytes through the data register (DR), and finally generates the STOP condition. Similarly, the I2C_MasterReceiveData() function performs blocking data reception. It supports both single-byte and multiple-byte reception by properly managing ACK control, clearing the ADDR flag, reading data from the data register, and generating the STOP condition when the transfer is completed.

While porting the driver to the STM32F401RE, the STM32F401RE device header file and board-specific definitions were used. The driver was configured for the I2C peripherals available on the STM32F401RE, and the timing parameters were calculated using the APB1

clock frequency of the board. The implemented APIs were tested by communicating with an I2C device and successfully performing both transmit and receive operations.

USART Driver Analysis

The USART driver is developed to provide a simple and reusable interface for serial communication between the STM32F401RE microcontroller and external devices such as computers, sensors, GPS modules, and Bluetooth modules. The driver avoids direct register access in the application program by providing API functions for USART initialization, data transmission, data reception, and interrupt-based communication. This improves code readability, portability, and simplifies the integration of USART peripherals in embedded applications.

The USART_PeriClockControl() function is used to enable or disable the clock supply for the USART peripheral through the RCC registers. The driver supports the USART peripherals available in STM32F401RE, where USART1 and USART6 are connected to the APB2 bus and USART2 is connected to the APB1 bus. The function identifies the selected USART instance and controls the corresponding peripheral clock.

The USART_Init() function configures the USART peripheral according to the user-defined settings. It enables the USART clock and configures important communication parameters such as USART mode, word length, parity control, stop bits, and hardware flow control. The function supports transmitter mode, receiver mode, and combined transmit-receive mode. It also allows configuration of 8-bit or 9-bit data frames and different parity options for reliable serial communication.

The USART_SetBaudRate() function calculates and configures the baud rate value in the USART Baud Rate Register (BRR). The baud rate calculation is performed based on the peripheral clock frequency and the selected communication speed. The function supports both oversampling by 16 and oversampling by 8 modes, and the calculated mantissa and fractional values are loaded into the BRR register to achieve accurate data transmission timing.

The driver includes functions such as USART_PeripheralControl() and USART_GetFlagStatus() for controlling and monitoring the USART operation. The peripheral control function enables or disables the USART by modifying the USART Enable (UE) bit in the CR1 register. The flag status function checks USART status flags such as TXE (Transmit Data Register Empty), TC (Transmission Complete), and RXNE (Receive Data Register Not Empty) to synchronize data transfer between the software and hardware.

The USART_SendData() function implements polling-based data transmission. It waits until the transmit data register becomes empty and then writes the data into the USART Data Register (DR). The function supports both 8-bit and 9-bit data transmission and handles parity configuration. After completing transmission, it checks the Transmission Complete (TC) flag to ensure that the entire data frame has been successfully transmitted.

The USART_ReceiveData() function performs polling-based data reception. It waits for the RXNE flag to indicate that new data is available, then reads the received data from the DR

register and stores it in the receive buffer. The function supports different word lengths and parity configurations based on the USART settings.

The driver also supports interrupt-based communication using USART_SendDataIT() and USART_ReceiveDataIT() functions. These functions enable USART interrupts such as TXE interrupt, Transmission Complete interrupt, and RXNE interrupt, allowing data transfer without continuous CPU polling. This improves processor efficiency by allowing other tasks to execute while communication is in progress.